Vulnerability Assessment: **Blind SQL Injection (Boolean / Timing-based)**
Project 4: Web Application Security Assessment
Week 3: Secure SDLC and Code Review

Vulnerability Information

- **Vulnerability Type:** Blind SQL Injection (Boolean / Timing-based)
- **Risk Level:** High
- **OWASP Category:** A03:2021 – Injection
- **Location:** /vulnerabilities/sqli/, lines 1–3

Vulnerable Code Analysis

- Low

```
1  $query  = "SELECT first_name, last_name FROM users WHERE user_id = '$id';";
```

- Medium

```
1  $query  = "SELECT first_name, last_name FROM users WHERE user_id = '$id';";
2
```

- High

```php
<?php

$query  = "SELECT first_name, last_name FROM users WHERE user_id = '$id';";

?>

```

• Direct concatenation of $id into SQL without parameterization.
• Application responses (content or timing) differ depending on query truth value, enabling boolean/timing inference.
• No uniform error handling; lack of explicit errors hides details but allows blind extraction via conditional queries.
• Inputs from GET/POST, cookies, headers, or sessions are trusted without validation or sanitization.

Impact
• Unauthorized data extraction one bit/character at a time (blind extraction).
• Disclosure of sensitive data (usernames, emails, hashes) without visible DB errors.
• Account compromise or information leakage when combined with other flaws.

Secure Code Example (PDO prepared statement + uniform response)

```php
// Validate & cast input
$id = isset($_GET['id']) ? filter_var($_GET['id'], FILTER_VALIDATE_INT) : false;
if ($id === false) {
    http_response_code(400);
    echo json_encode(['status'=>'error','message'=>'Invalid request.']);
    exit;
}

$pdo = new PDO($dsn, $db_user, $db_pass, [
    PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
    PDO::ATTR_EMULATE_PREPARES => false,
]);

$sql = "SELECT first_name, last_name FROM users WHERE user_id = :id LIMIT 1";
$stmt = $pdo->prepare($sql);
$stmt->bindValue(':id', $id, PDO::PARAM_INT);
$stmt->execute();
$row = $stmt->fetch(PDO::FETCH_ASSOC);

// Uniform response structure to avoid boolean leaks
$response = ['status' => 'ok', 'user' => null];
if ($row) {
    $response['user'] = [
        'first_name' => $row['first_name'],
        'last_name'  => $row['last_name']
    ];
}
echo json_encode($response);

```

Mitigations Applied

- Replaced string-concatenated queries with parameterized prepared statements.
- Implemented strict input validation and type-casting (whitelisting).
- Standardized uniform response format for present/absent data to prevent boolean inference.
- Disabled verbose database error messages to clients; errors logged server-side.
- Applied LIMIT and least-privilege database accounts.

Recommendations for Development Team
Immediate Actions:

1. **Parameterize All Queries**
   • Use prepared statements (PDO/mysqli) for every DB interaction; never concatenate user input.
2. **Validate & Whitelist Inputs**
   • Accept only expected types/formats (e.g., integers) and reject invalid inputs early.
3. **Uniform Response Handling**
   • Return consistent response structures and timings for success/failure to reduce information leakage.
4. **Suppress Detailed Errors**
   • Log detailed errors internally; present only generic errors to clients in production.
5. **Rate Limiting & Monitoring**
   • Implement request throttling and anomaly detection to hinder automated blind extraction attempts.
6. **Audit & Testing**
   • Run static analysis, automated scanners, and manual tests focused on blind (boolean/timing) SQLi patterns.

References
• OWASP: SQL Injection Prevention Cheat Sheet
• CWE-89: Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection')
• Best Practice: Validate, whitelist, and parameterize all inputs; use uniform responses and monitoring.